

CSE 575- Statistical Machine learning



Module 6

Samira Ghayekhlou, PhD
Assistant Teaching Professor @ ASU

Today's Agenda



- Tasks in module 6
- Explanation about module 6
-

Module 6 tasks



You need to submit the following tasks by Sunday April 26@11:59 pm

- Module 6: Graded Quiz
- Mini-Assignment : **Key Techniques for Deep Learning Mini-Assignment**



Explanation about module 6

Introduction to Neural Networks & Deep Learning

So far, we've learned how to predict labels (classification), values (regression), and group data (clustering). But what happens when the data is too complex for simple rules?

Quick Review:

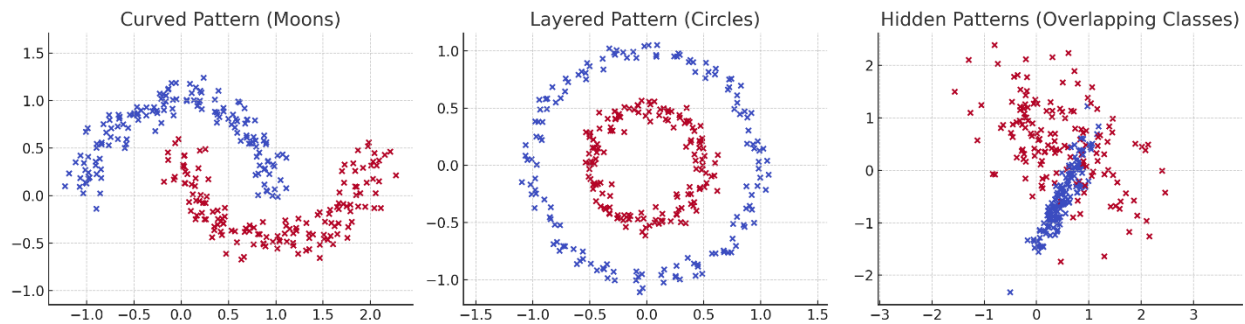
- **Regression:**
Predicting a continuous value using a linear model.
Example: $\text{Price} = w_1 * \text{size} + w_2 * \text{bedrooms} + b$
Limitation: Only works well when the relationship is approximately linear.
- **Classification:**
Predicting discrete labels. Logistic regression creates a decision boundary using a sigmoid function.
Limitation: Again, the boundary is linear.
- **K-Means (unsupervised):**
Finds clusters in data without labels.
Connection: Good for basic groupings, but struggles with high-dimensional, nonlinear patterns.

All of these models are **shallow models**; That means they make predictions based on a very small number of transformations on the input data, often just one layer of weights and bias.

Input → Linear function → Output

But real-world data is rarely that simple.

What if the relationship between input and output isn't a simple straight line? What if it's curved, involves interconnected or layered patterns, or reflects complex structures hidden deep within the data?



For example:

- Recognizing handwritten digits
- Detecting emotions in speech
- Identifying cancer from X-rays

These problems involve **high-dimensional, nonlinear** relationships that shallow models can't capture well.

Q: So, what can we use instead?

Let's introduce one of the most powerful tools in machine learning: the neural network.

Neural Network definition:

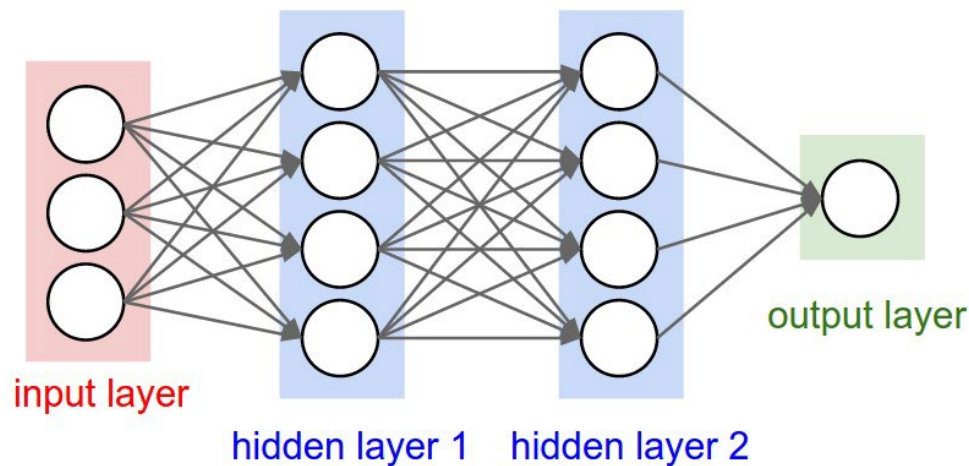
A neural network is a computational model inspired by the human brain, made up of layers of units called neurons. Each neuron performs a simple computation, and together, they can represent incredibly complex patterns.

Input → Hidden Layer (s) → Output

Each layer:

- Computes a weighted sum
- Adds a bias
- Applies an activation function
- Passes the result to the next layer

Example:



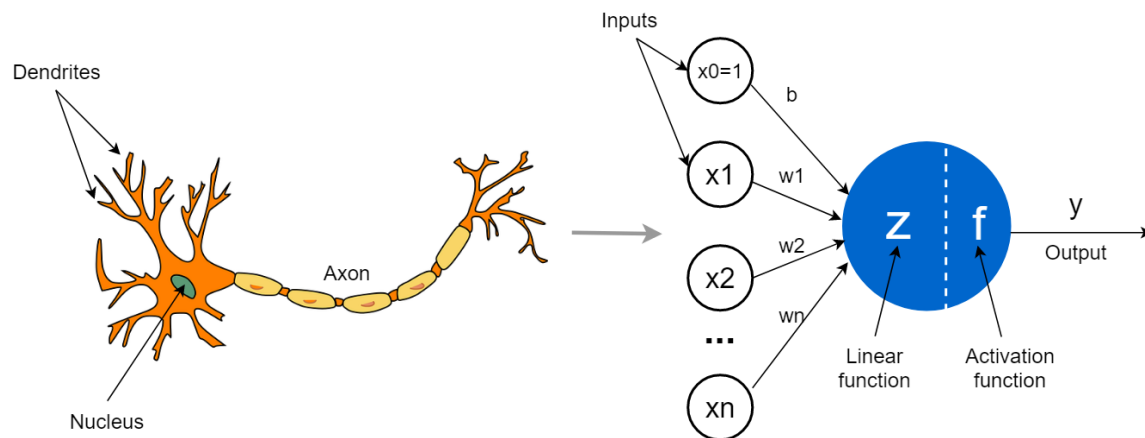
- **Input layer:** 3 features (x_1, x_2, x_3)
- **Hidden layer:** 2 neurons (h_1, h_2)
- **Output layer:** a single prediction (y)

By stacking multiple layers and using non-linear activation functions, we give the neural network the ability to model complex, high-dimensional relationships that shallow models can't handle.

Now that we have a basic idea of what a neural network is, let's start from the very beginning, with its most basic building block: the **perceptron**.

Perceptron Definition:

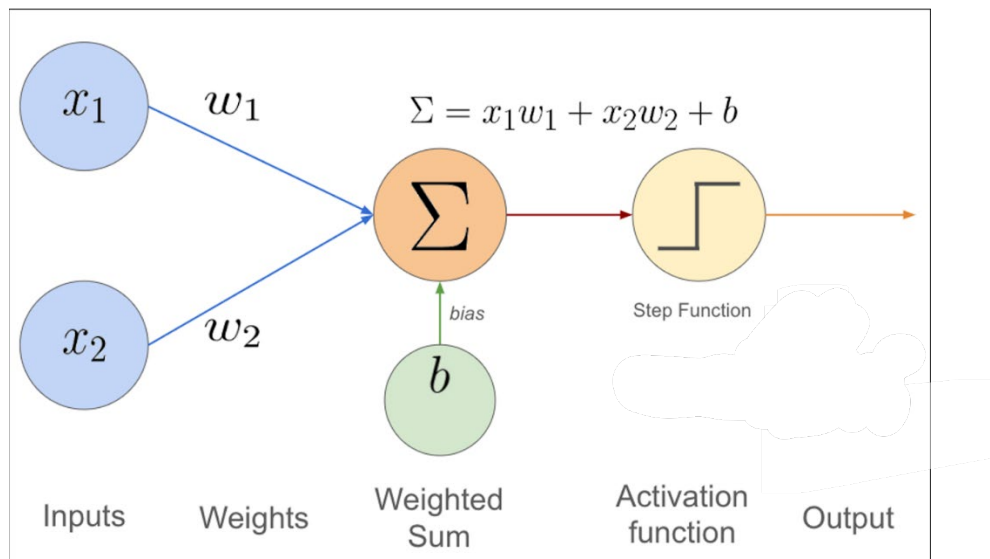
The perceptron is the **simplest neural network**, invented by **Frank Rosenblatt in 1958**. It mimics how a single neuron works in the brain.



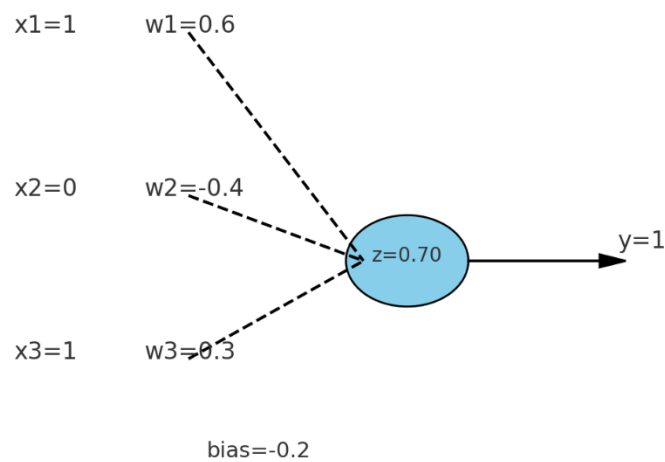
It takes a set of input features, multiplies them by weights, adds a bias, and applies an **activation (step) function** to make a decision, typically binary (yes/no or 0/1).

Activation (step) function:

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



Example:



Inputs: $x = [1, 0, 1]$

Weights: $w = [0.6, -0.4, 0.3]$

Bias: $b = -0.2$

The Perceptron computes the weighted sum:

$$z = (1 \times 0.6) + (0 \times -0.4) + (1 \times 0.3) + (-0.2) = 0.7$$

Activation (step) function: Since $z \geq 0$, the output $y = 1$

*****Limitation of Perceptron: Only Linearly Separable Data**

A perceptron can only solve problems where the classes can be separated by a **straight line** (in 2D), or a **hyperplane** in higher dimensions.

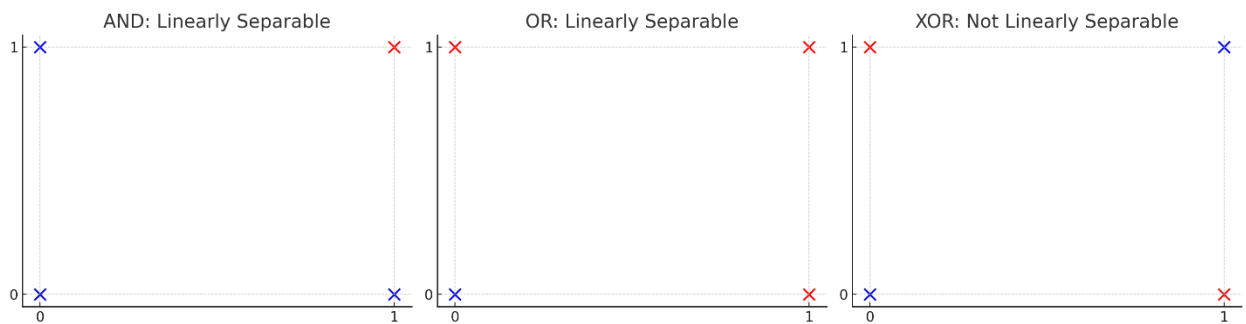
For example:

- **It can solve AND, OR**
- **But it fails on XOR** — which leads us to multi-layer networks.

● Red = Output 1

● Blue = Output 0

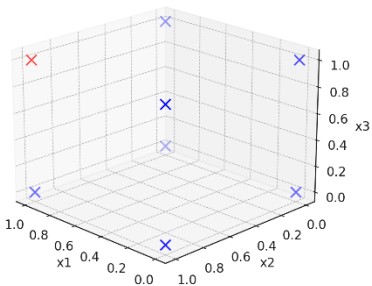
x1	x2	AND(x1, x2)	OR(x1, x2)	XOR(x1, x2)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



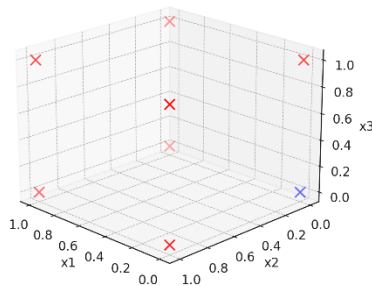
With more input:

x1	x2	x3	AND	OR	XOR
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	0	1	1
1	0	1	0	1	0
1	1	0	0	1	0
1	1	1	1	1	1

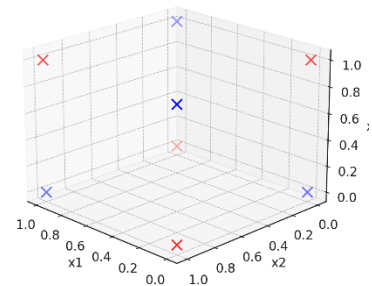
AND ($x1 \wedge x2 \wedge x3$)



OR ($x1 \vee x2 \vee x3$)



XOR (Odd number of 1s)

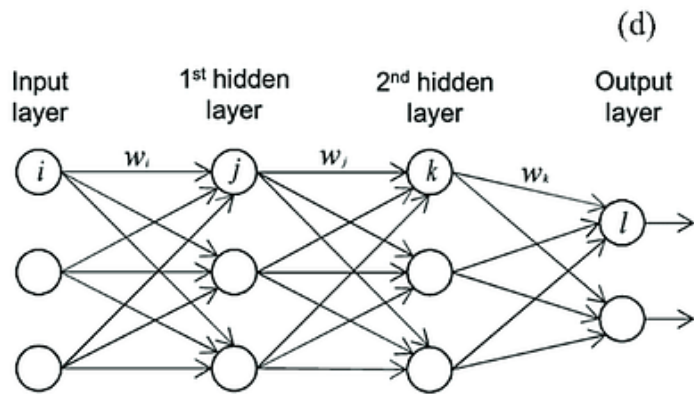
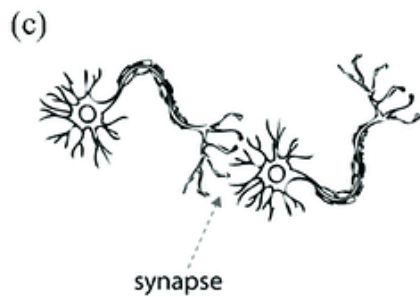
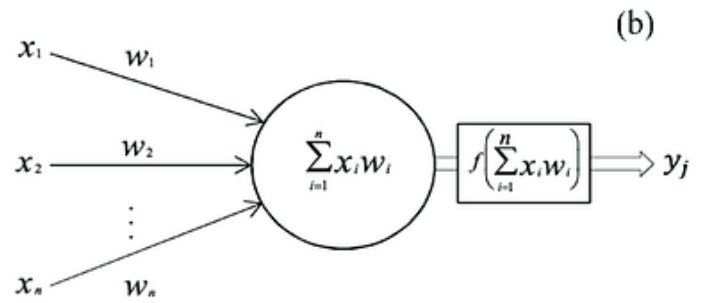
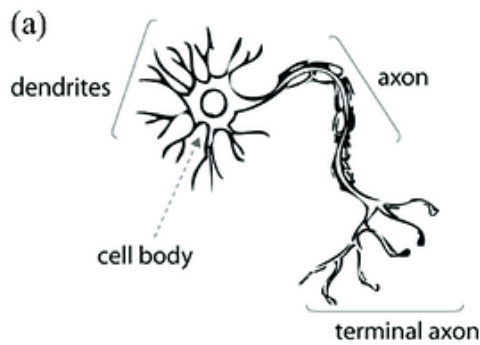


To go beyond linear separability, we need to add more layers , and that brings us to multi-layer perceptrons (MLP).

What is an MLP?

A Multi-Layer Perceptron (MLP) is simply a stack of perceptrons, multiple layers of neurons connected in sequence.

- A. Activation Functions
- B. Backpropagation



A. Activation Functions: Adding Non-Linearity

In a simple perceptron, the activation function (like the step function) is used just to make a binary decision, either yes or no, 0 or 1. It's like a switch.

But in Multi-Layer Perceptrons, the role of the activation function becomes much more powerful:

- The activation function decides **how the neuron responds**.
- It's not just for binary decisions anymore.
- Now, it enables the model to learn non-linear relationships between input and output.
- Without non-linearity, even if you stack 10 layers, the output is still a linear function of the input.

Common Activation Functions in MLPs:

1. Sigmoid

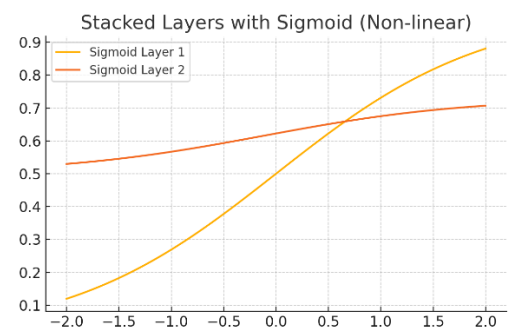
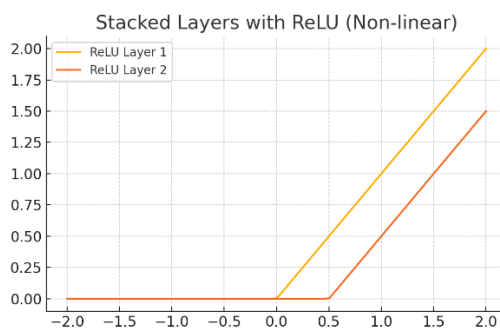
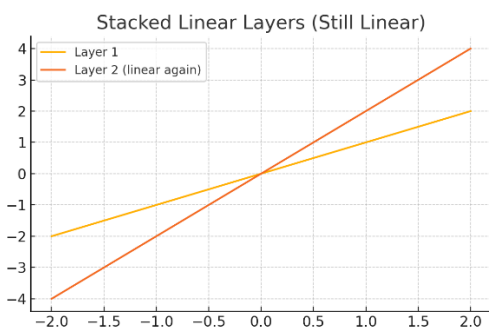
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Maps any input to a value between **0 and 1**
- S-shaped curve
- Sigmoid compresses everything into a probability-like output.

2. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- If $x < 0 \rightarrow \text{output} = 0$
- If $x > 0 \rightarrow \text{output} = x$
- ReLU keeps positive signals and completely shuts off negative ones.



Activation functions introduce non-linearity.

That non-linearity is what allows neural networks to learn complex patterns like images, language, and physics.

B. Backpropagation (Conceptual Overview)

Backpropagation is the learning algorithm used to train neural networks.
How should we change weight to reduce the error?

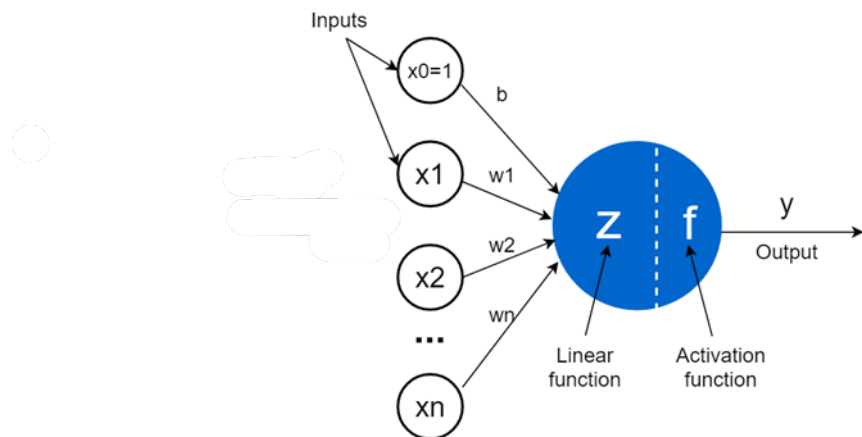
Neural networks learn by:

- Making a prediction
- Measuring the error
- Adjusting the weights to reduce that error

The Two Main Steps:

1. Forward Pass: (Forward pass is just computing the output.)

- Inputs go through the network layer by layer.
- At each layer:
 $z = w^t x + b$ then $a = \text{activation}(z)$
- We compute the final prediction $\hat{y}$.



2. Backward Pass (Backpropagation):

- Compare prediction $\hat{y}$ to actual value y .
- Use the loss function to calculate error.
- Then we send the error backward through the network to **adjust the weights**.

Each weight is changed a little, in the direction that would reduce the error.

The Loss Function

Explain that:

- Loss = a measure of how wrong the prediction was.
- Example: Mean Squared Error (MSE) for regression, or Cross-Entropy for classification.
- The network tries to **minimize this loss.**

Gradient Flow

Backpropagation uses a method called **gradient descent.**

It computes the gradient (slope) of the loss with respect to each weight, and updates the weights in the direction that reduces the loss.

We update each weight like this:

$$w := w - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

Where:

- η is the **learning rate** (how big the step is)
- $\frac{\partial \text{Loss}}{\partial w}$ is the gradient

Example of how NN working:

You are passing an input vector $x = [1, 0, 1]$ through an MLP that has:

- 3 input neurons
- 1 hidden layer with 2 neurons using ReLU
- 1 output neuron using Sigmoid

Step 1: Compute Hidden Layer Activations

Weights and Biases for Hidden Layer:

$$W_1 = \begin{bmatrix} 0.4 & -0.5 \\ 0.3 & 0.6 \\ 0.7 & -0.1 \end{bmatrix}, \quad b_1 = [0.2 \quad -0.4]$$

Input vector:

$$x = [1 \quad 0 \quad 1]$$

Weighted sum:

We compute:

$$z_1 = x \cdot W_1 + b_1$$

Break it down:

- Neuron 1:

$$z_{1,1} = 1 * 0.4 + 0 * 0.3 + 1 * 0.7 + 0.2 = 1.3$$

- Neuron 2:

$$z_{1,2} = 1 * (-0.5) + 0 * 0.6 + 1 * (-0.1) - 0.4 = -1.0$$

So:

$$z_1 = [1.3, -1.0]$$

Apply ReLU activation:

$$a_1 = \text{ReLU}(z_1) = [\max(0, 1.3), \max(0, -1.0)] = [1.3, 0]$$

Step 2: Compute Output Layer Activation

Weights and Bias:

$$W_2 = [0.9 \quad -1.1], \quad b_2 = -0.3$$

Input from hidden layer:

$$a_1 = [1.3, 0]$$

Weighted sum:

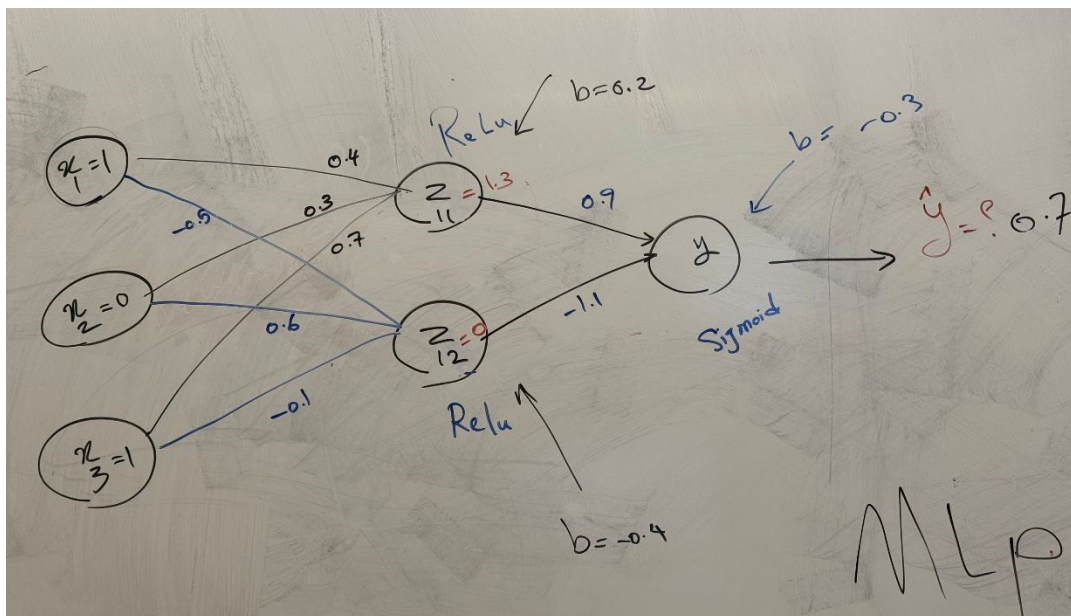
$$z_2 = a_1 \cdot W_2 + b_2 = 1.3 * 0.9 + 0 * (-1.1) - 0.3 = 1.17 - 0.3 = 0.87$$

Apply Sigmoid:

$$\hat{y} = \sigma(0.87) = \frac{1}{1 + e^{-0.87}} \approx 0.70$$

The predicted value is:

$$\hat{y} \approx 0.70$$





Q & A